Taller 14 - Árboles de Decisión y Aprendizaje Supervisado

Brayan Camilo Moreno Villalobos

SECCIÓN AJUSTE

A. Tasa de deserción y exactitud del modelo "ingenuo"

```
tasa_desercion <- prop.table(table(desercion$deserta))
print(tasa_desercion)
```

```
       No        Sí 
0.7841667 0.2158333 
```

La tasa de deserción corresponde a la proporción de la clase "Sí". Si un modelo predice siempre "No" (la clase mayoritaria), su exactitud (accuracy) será exactamente igual a la proporción de "No" en la base de datos (por ejemplo, si los "No" son el 78%, la exactitud será del 78% sin necesidad de aprender ninguna regla).

B. La variable que NO estaría disponible al momento de predecir es retiro_registrado. Esta variable es una consecuencia directa o un registro administrativo que ocurre después de que el estudiante ya desertó. Incluirla causaría "fuga de información" (data leakage) y haría que el árbol sea inútil para prevenir la deserción en la vida real. La excluiremos del modelo.

c.

```
library(caret)
library(rpart)
library(rpart.plot)
datos_modelo <- desercion[, !names(desercion) %in% "retiro_registrado"]

# Partición 70/30
set.seed(2026)
train_idx <- createDataPartition(datos_modelo$deserta, p = 0.7, list = FALSE)
train_data <- datos_modelo[train_idx, ]
test_data <- datos_modelo[-train_idx, ]

# 1. Árbol sin restricciones (cp = 0, minsplit = 2)
arbol_full <- rpart(deserta ~ ., data = train_data, method = "class",
                    control = rpart.control(cp = 0, minsplit = 2))

# 2. Árbol podado con validación cruzada
# Encontramos el CP óptimo que minimiza el error cruzado (xerror)
cp_optimo <- arbol_full$cptable[which.min(arbol_full$cptable[,"xerror"]),"CP"]
```

```r
arbol_podado <- prune(arbol_full, cp = cp_optimo)

# Comparación de hojas
hojas_full <- sum(arbol_full$frame$var == "<leaf>")
hojas_podado <- sum(arbol_podado$frame$var == "<leaf>")
cat("Hojas árbol sin restricciones:", hojas_full, "\n")
```

```
Hojas árbol sin restricciones: 149
```

```r
cat("Hojas árbol podado:", hojas_podado, "\n")
```

```
Hojas árbol podado: 4
```

```r
# Exactitud en Entrenamiento y Prueba
pred_full_train <- predict(arbol_full, train_data, type = "class")
pred_full_test <- predict(arbol_full, test_data, type = "class")
pred_podado_train <- predict(arbol_podado, train_data, type = "class")
pred_podado_test <- predict(arbol_podado, test_data, type = "class")

acc_full_train <- mean(pred_full_train == train_data$deserta)
acc_full_test <- mean(pred_full_test == test_data$deserta)
acc_podado_train <- mean(pred_podado_train == train_data$deserta)
acc_podado_test <- mean(pred_podado_test == test_data$deserta)

resultados <- data.frame(
  Modelo = c("Sin restricciones", "Podado"),
  Acc_Train = c(acc_full_train, acc_podado_train),
  Acc_Test = c(acc_full_test, acc_podado_test)
)
print(resultados)
```

```
             Modelo Acc_Train  Acc_Test
1 Sin restricciones 1.0000000 0.7910864
2            Podado 0.8466112 0.8356546
```

El árbol sin restricciones genera demasiadas hojas (sobreajuste), logrando una exactitud casi perfecta en entrenamiento, pero menor en prueba. El árbol podado reduce drásticamente el número de hojas, generalizando mejor y obteniendo una exactitud similar o superior en la base de prueba, evitando el sobreajuste.

SECCIÓN INTERPRETACIÓN

D. Reglas del árbol podado

```r
# Visualizamos las reglas del árbol podado
rpart.rules(arbol_podado, cover = TRUE)
```

```
 deserta                                                          cover
    0.13 when papa >= 3.3                                          83%
    0.29 when papa <  3.3 & creditos_perdidos <  3 & semestre <  8  3%
    0.63 when papa <  3.3 & creditos_perdidos <  3 & semestre >= 8  2%
    0.74 when papa <  3.3 & creditos_perdidos >= 3                 12%
```

Las reglas impresas por el código anterior muestran las particiones clave y el porcentaje de datos (cobertura) que cae en cada nodo terminal.

1. E. Revisando las reglas del árbol podado (rpart.rules), verificamos si recupera las condiciones originales de generación:
   papa $<$ 3.3 y creditos_perdidos
2. $>=$ 3papa $<$ 3.3horas_trabajo
3. $>=$ 24 y estrato $<=$ 2

Generalmente, el árbol recuperará perfectamente las reglas relacionadas con el papa, ya que tienen las probabilidades de deserción más altas (75% y 40%). Sin embargo, la regla de horas_trabajo y estrato podría quedar oculta o fusionada si la cantidad de datos que cumplen esa condición exacta es muy baja para justificar una nueva rama durante la poda.

F. La variable `distancia_km` **no aparece** en el árbol podado. Esto significa que el algoritmo de partición recursiva y la validación cruzada funcionaron correctamente: identificaron que esta variable es "ruido" y no aporta ganancia de información real, ya que en nuestro código de simulación la distancia no afectaba en absoluto a la probabilidad `p` de desertar.

G.

```
# Cargar caret (necesario para confusionMatrix)
library(caret)

# 1. Convertir a factores con los mismos niveles
pred_podado_test_factor <- factor(pred_podado_test, levels = c("No", "Sí"))
test_data_deserta_factor <- factor(test_data$deserta, levels = c("No", "Sí"))

# 2. Sensibilidad actual
conf_matrix_podado <- confusionMatrix(data = pred_podado_test_factor,
                                      reference = test_data_deserta_factor,
                                      positive = "Sí")
sensibilidad_actual <- conf_matrix_podado$byClass["Sensitivity"]
cat("Sensibilidad actual:", sensibilidad_actual, "\n")
```

```
Sensibilidad actual: 0.3636364
```

```
# 3. Ajustar un árbol con matriz de costos (Penalizamos Falsos Negativos)
matriz_costos <- matrix(c(0, 1,
                          5, 0), byrow = TRUE, nrow = 2)

arbol_costos <- rpart(deserta ~ ., data = train_data, method = "class",
                      parms = list(loss = matriz_costos),
                      control = rpart.control(cp = cp_optimo))

# 4. Predicciones con costos y conversión a factor
pred_costos_test <- predict(arbol_costos, test_data, type = "class")
pred_costos_test_factor <- factor(pred_costos_test, levels = c("No", "Sí"))

# 5. Sensibilidad con costos
conf_matrix_costos <- confusionMatrix(data = pred_costos_test_factor,
                                      reference = test_data_deserta_factor,
                                      positive = "Sí")
```

```r
sensibilidad_costos <- conf_matrix_costos$byClass["Sensitivity"]
cat("Sensibilidad con costos:", sensibilidad_costos, "\n")
```

```
Sensibilidad con costos: 0.7532468
```

Al introducir pesos o matrices de costos que penalizan los falsos negativos, obligamos al árbol a predecir "Sí" con mayor facilidad. Esto **aumenta la sensibilidad** (capacidad de detectar a los posibles desertores)[cite: 23]. ¿Qué se pierde? Se pierde **Especificidad** y, usualmente, Exactitud global[cite: 23]. Esto se traduce en más **Falsos Positivos**: el modelo clasificará como potenciales desertores a muchos estudiantes que en realidad se iban a quedar.